
PyTorch Part 2

Devices, Precision & the Training Toolkit

Block 5

From Data to Decisions

Today

Part 1 built the model: tensors, the forward pass, autograd, the loop. It *runs*. Part 2 is everything **around** it: making training fast, correct, reproducible, and observable.

1 Devices: CPU vs GPU; moving work

2 Autograd & inference: build the graph, or not

3 Mixed precision: more speed, fewer bits

4 einops: readable tensor reshaping

5 Reproducibility: same run, same numbers

6 Tracking: make training observable

7 Scaling out: a first look at multi-GPU

Devices

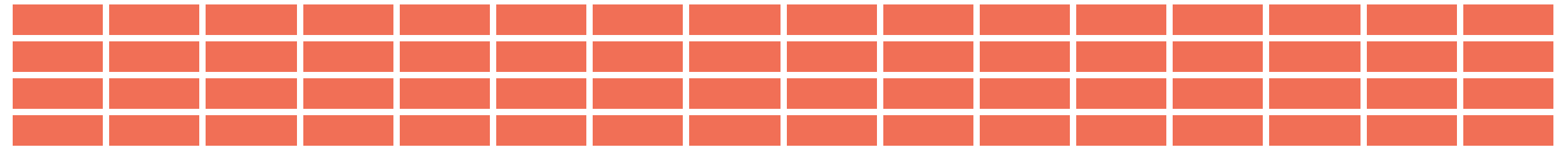
where your tensors live, and why it matters

Why a GPU is fast

A **CPU** has a few powerful cores optimized for **latency**: finishing one complex task as fast as possible. A **GPU** has thousands of simple cores optimized for **throughput**: running the same simple operation on huge amounts of data in parallel.



CPU: a few fast cores



GPU: thousands of small cores

Why it matters for training

A training step is mostly one operation, a matrix multiply, repeated across the whole batch. Identical math, massively parallel: precisely the workload a GPU is built for.

Two separate memories: RAM and VRAM

CPU and GPU don't share memory. Your data starts in system **RAM** (the *host*); the GPU computes only on its own **VRAM** (the *device*). The two are joined by the **PCIe bus**, fast, but far slower than either chip's own memory.



Where does a tensor live?

Every tensor and module reports its **.device**. Two tensors on different devices can't be combined directly, move one first, or PyTorch raises an error.

Moving work to the device

`.to(device)` copies a tensor or a whole module between RAM and VRAM. Two habits:

- 1 It returns a **new copy** for tensors: `.to` is not in-place. Always assign: `x = x.to(device)`. It **does** modify **in-place** when called on a **module**. Here, you can just call: `model.to(device)`
- 2 Move the **model and every batch**: data is born on the CPU each step and must meet the weights on the same device.

```
device = "cuda" if torch.cuda.is_available() else "cpu"

model.to(device) # weights → VRAM

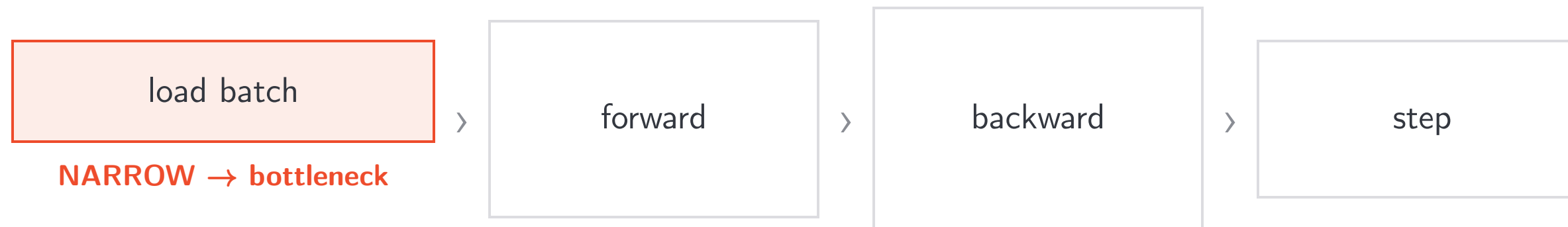
for x, y in loader:
    x = x.to(device) # batch → VRAM
    y = y.to(device) # Note: tensors not modified in-place
    loss = loss_fn(model(x), y)
```

The cost is real but small

Copying bytes across the bus isn't free, but it's negligible next to running the actual math on the right hardware. Pay it once per batch, not once per operation.

A pipeline is only as fast as its narrowest stage

Training throughput is a **pipeline**: load a batch → move to device → forward → backward → step. Like water through pipes, the **narrowest segment sets the speed** of the whole run.



GPU starvation

A fast GPU idling while the CPU preps the next batch is the No. 1 cause of "slow training". If measured time $\gg$ the math alone, the **data path** is the narrow stage. Fixes: precompute & cache heavy transforms, add DataLoader workers, prefetch.

Autograd & Inference

when to build the graph, and when not to

Autograd records the whole graph

In the forward pass, autograd records every operation into a **computation graph** and keeps the **intermediate tensors**. **backward()** walks that graph in reverse, applying the chain rule to fill each parameter's **.grad**.



The cost

Those stored intermediates are what make training possible, and what make a forward pass with grad tracking **on** cost far more memory than the computation alone.

Inference doesn't need the graph

At inference there's **no backward pass**, so building the graph and keeping intermediates is pure waste. Switch it off:

- › **`torch.no_grad()`**: stop recording the graph.
- › **`torch.inference_mode()`**: newer, stricter, even faster; the default for serving.

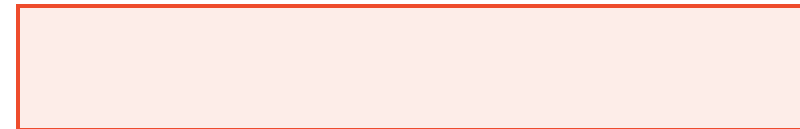
```
model.eval()
with torch.inference_mode():
    logits = model(x.to(device))
preds = logits.argmax(dim=1)
```

peak memory, one forward pass

grad tracking ON



inference_mode



No stored intermediates → a fraction of the memory → more concurrent requests per GPU.

eval() and no_grad() are different switches

They're **orthogonal**: one changes layer behaviour, the other controls autograd. Inference needs both.

`model.eval()`

changes LAYER behaviour

- › dropout turns **off**
- › batchnorm uses **running stats**, not the batch's
- › affects the **correctness** of outputs

`torch.no_grad()`

controls AUTOGRAD

- › stops the graph being built
- › frees the stored intermediates
- › affects **memory & speed**

Use both for inference

eval() alone still records a graph (wasted memory); **no_grad()** alone leaves dropout active (wrong outputs).

Freezing parameters

Set `requires_grad = False` to **freeze** a parameter: it leaves the gradient graph and the optimizer never updates it.

Reuse a pretrained **backbone** as-is; let only the new **head** learn. This is the core move in **transfer learning** and fine-tuning.

```
# freeze the pretrained backbone
for p in model.backbone.parameters():
    p.requires_grad = False

# only trainable params reach the optimizer
optim = torch.optim.AdamW(
    (p for p in model.parameters()
     if p.requires_grad), lr=1e-3)
```

Why freeze

No gradients for frozen layers → less compute and memory. And features learned on a much larger dataset are protected from being washed out by a small one.

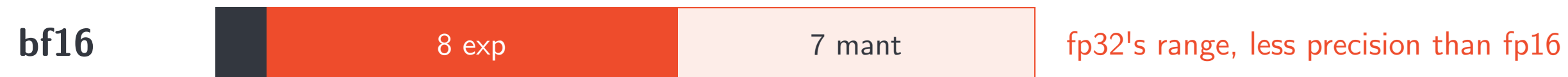
Mixed Precision

more throughput, fewer bits

fp32, fp16, bf16: the bit budget

Exponent bits set the **range** (how big / small); **mantissa** bits set the **precision** (how fine). fp16 and bf16 are both 16 bits, they just spend them differently.

■ sign ■ exponent: range ■ mantissa: precision



Does fp16 actually run faster?

Half the **memory** is always real. Half the **time** is not, it depends on whether the GPU has hardware that runs 16-bit math faster.

A · no fp16 units

Pads each fp16 back to fp32 and does fp32 math. **No speedup.**

B · packs two per unit

Two fp16 through one fp32 unit per cycle.
Approx. 2× faster.

C · dedicated units

Tensor-Core-style 16-bit units run faster than fp32 ones. **Real speedup.**

Takeaway

Fewer **bytes** is guaranteed. Fewer **nanoseconds** is a hardware question, on modern GPUs with Tensor Cores, the answer is yes.

The fp16 trap: vanishing gradients

fp16's narrow range has a floor near 6×10^{-8} . Gradients are often tiny; below that floor they round to **exactly 0**, and a zero gradient means that weight **stops learning**.



A tiny experiment

Take a gradient of **1.0** and halve it 26 times (as deep layers do). In fp32 it's still **$1.5e-8$** , alive. In fp16 it has already underflowed to **0**.

Two cures: loss scaling and bf16

Loss scaling (GradScaler)

Multiply the loss by a big factor **before** `backward()`, gradients lift out of the underflow zone; **unscale** just before the step. **GradScaler** tunes the factor automatically: up when safe, down on overflow.

bf16

Same 16 bits, laid out with **fp32's range** (8 exponent bits). Tiny gradients simply don't underflow, **no scaler needed**. The cost is reduced precision.

Rule of thumb

bf16 if your hardware supports it, it's the simpler switch. Otherwise **fp16 + GradScaler**. (Even in bf16, keep the final loss & reductions in fp32 for stability.)

Mixed precision in practice

autocast runs each op in the right dtype automatically, fast 16-bit for matmuls & convs, fp32 where precision matters. You never cast by hand.

```
scaler = torch.cuda.amp.GradScaler()           # only for fp16

for x, y in loader:
    x, y = x.to(device), y.to(device)
    optim.zero_grad()
    with torch.autocast("cuda", dtype=torch.float16):
        loss = loss_fn(model(x), y)           # autocast picks per-op dtype
    scaler.scale(loss).backward()              # lift gradients
    scaler.step(optim); scaler.update()         # apply + retune the factor
```

Switch **dtype=torch.bfloat16** and you can drop the scaler entirely, call **loss.backward()** and **optim.step()** as usual.



name your axes

Every einops pattern is a sentence

A pattern is just a string: **"input axes -> output axes"**. Read it literally.

`rearrange(x, "b c h w -> b (c h w)")`

names every input axis, in order

the output, built from those names

- › **Left of** → names every input axis (**b c h w**).
- › **Right of** → describes the output you want.
- › **Reorder** the names → a permute.
- › **() on output** merges axes into one.
- › **() on input** splits one axis into several.
- › Merging is an **odometer**: rightmost ticks fastest.

Three verbs cover almost everything

verb	what it does	raw PyTorch it replaces
rearrange	move / merge / split / permute axes	<code>view</code> , <code>reshape</code> , <code>permute</code> , <code>transpose</code> , <code>flatten</code>
reduce	collapse axes with an aggregation	<code>x.mean(dim=...)</code> , <code>x.sum(dim=...)</code> , <code>x.max(...)</code>
repeat	add or expand axes by copying	<code>expand</code> , <code>repeat</code> , <code>tile</code>

Why bother

view / **permute** / **reshape** say **what** shape, never **which** axis goes where. einops names the axes, so the line reads as the operation, and wrong orders stop being silent.

rearrange: merge, split, permute

Each line beside the raw-PyTorch twin it replaces, same result, the intent now visible.

```
rearrange(x, "b c h w -> b (c h w)")      # flatten  == x.view(b, -1)
rearrange(x, "b h w c -> b c h w")        # permute   == x.permute(0,3,1,2)
rearrange(x, "b (n d) -> b n d", n=3)     # split    == x.reshape(b,3,-1)
```

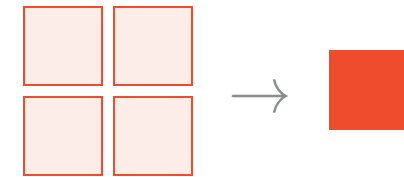
Read the twins

`.view(b, -1)` hides which axes merged and in what order; `.permute(0,3,1,2)` is a puzzle of indices. The named patterns say it out loud.

reduce: the pattern is the formula

The axes that **disappear from the right** are exactly the ones collapsed. Global average pooling, in one line:

```
pooled = reduce(x, "b c h w -> b c", "mean")    # == x.mean(dim=(2,3))  
pooled = reduce(x, "b c h w -> b c", "max")     # == x.amax(dim=(2,3))
```



each $h \times w$ plane $\rightarrow$ one number per channel

Order is everything

Same letters, **different order** → different result. Naming makes the difference very hard to miss.

```
x.shape                                # (batch=2, tokens=3, dim=4)

rearrange(x, "b n d -> b (n d)")      # (2, 12) batch stays batch ✓
rearrange(x, "b n d -> n (b d)")      # (3, 8) batch mixed in!   ✗
```

The real-world bug

A wrong **.view()** that happens to give the right **shape** runs silently, scrambling which features belong to which sample, so the model never learns. The highest-leverage habit in this deck: **name your axes**.

Reproducibility

same run, same numbers

Sources of randomness

A run varies because many steps draw on an **RNG**. To reproduce a result, pin them all:

- › weight initialization
- › DataLoader shuffling
- › non-deterministic GPU kernels
- › data augmentation
- › dropout / stochastic layers
- › parallel-reduction order

```
torch.manual_seed(0)
np.random.seed(0)
random.seed(0)
torch.use_deterministic_algorithms(True)
```

Caveat to the rule

Even seeded, **bit-for-bit** reproducibility also depends on hardware, library versions, and cuDNN. Seeds get you **repeatable**; full determinism costs some speed. Particularly, the last line of code above is **rarely** used.

What carries state?

Not just the weights. Several things **evolve during training** and would be silently reset on resume:

- ✓ model **parameters & buffers** (e.g. batchnorm running stats)
- ✓ **optimizer** state: AdamW's per-parameter moments
- ✓ **epoch / global step**
- ✓ LR **scheduler** state
- ✓ AMP **GradScaler**: the current loss-scale factor
- ✓ **RNG state**, so shuffling & dropout continue identically

The non-obvious ones

AdamW's **moments**, the scaler's **scale**, and the **RNG** don't look like the model, but drop any of them and the first steps after resume lurch while they rebuild from scratch.

A checkpoint is a resumable snapshot

Save **everything that carries state**, then resume as if nothing happened.

```
torch.save({
    "model":      model.state_dict(),
    "optimizer": optim.state_dict(),
    "scaler":     scaler.state_dict(),
    "epoch":      epoch,
    "rng":        torch.get_rng_state(),
}, "ckpt.pt")

ckpt = torch.load("ckpt.pt", map_location=device)
model.load_state_dict(ckpt["model"])
optim.load_state_dict(ckpt["optimizer"])      # ...restore them all
```

Broken vs complete

Saving **only** `model.state_dict()` is the classic bug, it resumes with random optimizer momentum and a reset RNG. **How often** to save is a trade-off: more frequent = less lost on a crash, but each save pauses training.

Tracking & Visualization

make the problem visible

You can't fix what you can't see

You can't tell over- from under-fitting unless you track **train** and **validation** loss together. Three shapes:



Read the axis honestly

Plot tools auto-zoom, so a flat line can look like progress. Pin the y-axis to **0** and draw the **random-guess baseline** (a 2-class model that learned nothing sits at $\ln 2 \approx 0.69$).

TensorBoard and Weights & Biases

TensorBoard

Local, ships with PyTorch. Scalars, histograms, images, the graph. Ideal on a single machine.

```
w = SummaryWriter()  
w.add_scalar("loss/train", loss, step)
```

Weights & Biases

Cloud experiment tracking. Dashboards across runs, hyperparameter sweeps, artifacts, team sharing.

```
wandb.init(project="churn")  
wandb.log({"loss/train": loss}, step=step)
```

What to log

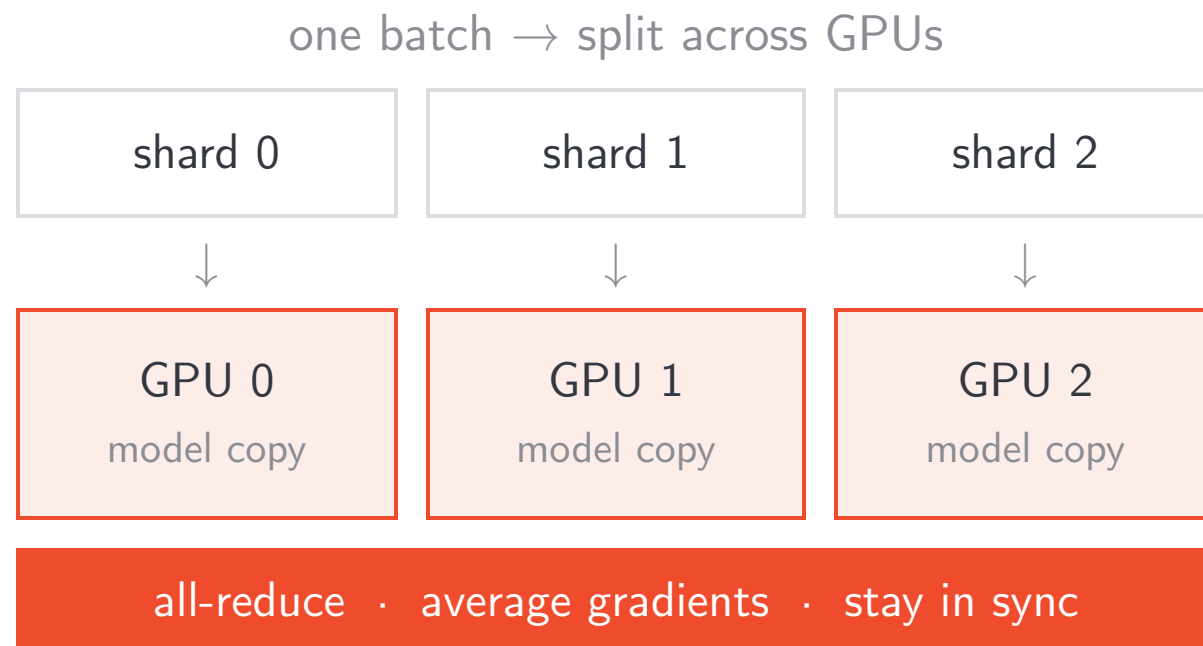
Scalars (loss, accuracy, LR) · **distributions** (weight / gradient histograms) · **media** (sample predictions) · and your full **config**, so a run is reproducible and comparable.

Scaling Out

a first look at multi-GPU

Beyond one GPU: data parallelism & Accelerate

The first lever is **data parallelism**: replicate the model on every GPU, split each batch across them, and **average the gradients** (all-reduce) so every replica stays in sync.



```
from accelerate import Accelerator
acc = Accelerator(mixed_precision="bf16")

model, optim, loader = acc.prepare(
    model, optim, loader)

for x, y in loader:
    loss = loss_fn(model(x), y)
    acc.backward(loss)          # grad sync
    optim.step(); optim.zero_grad()
```

Same loop on 1 GPU or many. Bigger than memory? **Sharding** (FSDP / ZeRO) splits the model itself, next session.

The training toolkit

Devices	<code>.to(device)</code> everything; GPU for throughput; watch for a starving GPU
Autograd & inference	graph for training; <code>eval()</code> + <code>inference_mode()</code> for serving; freeze to fine-tune
Mixed precision	16-bit for speed & memory; <code>autocast</code> + GradScaler (fp16) or bf16
einops	name your axes: readable reshapes, fewer silent bugs
Reproducibility	seed every RNG; checkpoint all state, not just weights
Tracking	TensorBoard / W&B; track train vs validation on an honest axis
Scaling out	data parallelism + Accelerate; sharding (FSDP / ZeRO) for huge models

The throughline: make it visible, find the narrow stage, and fix *that*, instead of throwing more compute at a pipe that's blocked somewhere else.